

Trabajo Integrador de Aplicaciones Interactivas

El Trabajo Práctico Obligatorio (TPO) consiste en el desarrollo progresivo de una **aplicación web transaccional con funcionalidades de e-commerce**, implementada mediante una arquitectura cliente-servidor.

El proyecto se desarrollará en **dos etapas obligatorias**, correspondientes a dos entregas incrementales. Cada etapa deberá constituir un sistema funcional y verificable, y deberá servir como base para la siguiente.

El trabajo podrá realizarse individualmente o en grupos de **2 a 4 integrantes**.

La evaluación será **individual**, independientemente de la modalidad grupal. Cada integrante deberá poder explicar y justificar la arquitectura, tecnologías, patrones de diseño, estructuras de datos, decisiones de implementación y funcionamiento de las partes desarrolladas.

ETAPA 1 — BACK-END Y PERSISTENCIA

Objetivo:

Desarrollar el **back-end de la aplicación**, implementando la lógica de negocio, persistencia de datos y una interfaz de comunicación mediante red que permita al futuro front-end interactuar con el sistema.

La aplicación deberá ejecutarse inicialmente en un entorno local mediante **localhost**, pero deberá estar diseñada bajo una arquitectura **cliente-servidor**, evitando soluciones que dependan exclusivamente de una ejecución local o de una comunicación directa entre la interfaz y la base de datos.

Tecnologías:

- **Java.**
- **Framework establecido por la cátedra.** (Spring)
- **SQL** y un sistema gestor de base de datos.
- Comunicación mediante **HTTP**.
- API de comunicación entre cliente y servidor.
- Formato de intercambio de datos adecuado.

Durante la entrega deberá demostrarse el funcionamiento del back-end mediante herramientas que permitan realizar solicitudes HTTP.

Debe estar acompañado por una documentación que explique el proceso de desarrollo del ecosistema planteado. Con detalles exhaustivos y bien definidos sobre que hace cada cosa en su sistema.

- Identificación de capas en la estructura de software
- Conocimiento de las responsabilidades de las clases planteadas.
- Explicación sobre la sintaxis de los elementos requeridos.
- Justificación sobre las decisiones de diseño aplicadas.

Requerimientos:

El sistema deberá implementar, como mínimo:

1. **Arquitectura organizada por capas o responsabilidades.**
2. Conexión y persistencia mediante una base de datos SQL.
3. Modelado de las entidades principales del dominio.
4. Operaciones **CRUD** sobre las entidades correspondientes.
5. Implementación de la lógica de negocio.
6. Exposición de endpoints para permitir la comunicación con clientes externos.
7. Manejo adecuado de solicitudes HTTP y respuestas.
8. Validación de datos de entrada.
9. Manejo de errores y respuestas apropiadas.
10. Configuración que permita ejecutar el servidor mediante localhost.
11. Documentación básica de los endpoints disponibles.
12. Separación entre la lógica de negocio, acceso a datos y capa de comunicación.
13. El sistema deberá poder ejecutarse y responder solicitudes en tiempo real.

Resultado esperado:

Se debe contar con un servidor capaz de recibir solicitudes externas, procesarlas, ejecutar la lógica correspondiente y consultar/modificar información persistida en la base de datos.

El front-end definitivo **no forma parte de esta entrega.**

ETAPA 2 — INTEGRACIÓN FRONT-END + BACK-END

Objetivo:

Desarrollar e integrar la **interfaz de usuario**, utilizando el back-end desarrollado durante la primera etapa como servidor de la aplicación.

El objetivo de esta etapa es transformar el back-end previamente desarrollado en una **aplicación web funcional e interactiva**, accesible mediante una interfaz gráfica.

Tecnologías mínimas

El front-end deberá utilizar:

- **HTML5.**
- **CSS3.**
- **JavaScript.**
- Comunicación con el back-end mediante **HTTP**.
- Intercambio de información mediante el formato definido para la API.

La aplicación deberá:

1. Implementar una interfaz web funcional.
2. Consumir los endpoints desarrollados durante la primera etapa.
3. Mostrar información obtenida desde la base de datos.
4. Permitir al usuario realizar operaciones sobre el sistema.

5. Implementar validaciones en la interfaz.
6. Gestionar adecuadamente estados de carga y errores.
7. Mantener una separación clara entre presentación y lógica del servidor.
8. Utilizar la API del back-end para las operaciones de datos.
9. Implementar las funcionalidades principales definidas para el dominio seleccionado.
10. Ser demostrable en tiempo real.

Condición fundamental del TPO

La aplicación **no deberá quedar planteada como una solución exclusivamente local o monolítica.**

Aunque durante las primeras etapas pueda ejecutarse mediante localhost, la arquitectura deberá permitir que el cliente y el servidor se comuniquen mediante protocolos de red y que, mediante la configuración correspondiente, el servidor pueda ser trasladado a otro equipo o entorno de ejecución.

La comunicación deberá respetar una arquitectura cliente-servidor: Front-end → HTTP → Back-end → Base de datos.

Evaluación

La evaluación contemplará tanto el **resultado funcional** como la comprensión individual del desarrollo.

Cada integrante podrá ser evaluado sobre:

- arquitectura;
- patrones de diseño;
- principios de desarrollo de software;
- estructura del proyecto;
- funcionamiento del framework;
- comunicación HTTP;
- diseño y consumo de API;
- persistencia SQL;
- lógica de negocio;
- integración front-end/back-end;
- decisiones técnicas adoptadas;
- capacidad para modificar, extender o justificar el código desarrollado.

La pertenencia a un grupo no implica que todos los integrantes reciban automáticamente la misma calificación.

Preguntas de guía para ir completando a medida que avanza el proyecto:

Arquitectura

- ¿Por qué dividieron el proyecto en estas capas?
- ¿Qué responsabilidad tiene cada capa?
- ¿Qué ocurriría si colocáramos toda la lógica en el Controller?

- ¿Dónde debería estar una regla de negocio y por qué?
- ¿Qué componente debería modificarse si cambiamos el motor de base de datos?

MVC

- ¿Dónde está el Model en su aplicación?
- ¿Cuál es la responsabilidad del Controller?
- ¿Qué diferencia hay entre Controller y Service?
- ¿Por qué la View no debería acceder directamente a la base de datos?

DAO

- ¿Qué problema resuelve DAO?
- ¿Qué responsabilidad tiene?
- ¿Qué ocurriría si elimináramos esta capa?
- ¿DAO y ORM son lo mismo?

Esta última es excelente para detectar aprendizaje real.

ORM

- ¿Qué significa Object-Relational Mapping?
- ¿Qué relación existe entre una Entity y una tabla?
- ¿Qué representa una relación @OneToMany?
- ¿Qué ventajas tiene utilizar ORM?
- ¿Qué problemas puede generar una mala utilización del ORM?

SQL

- ¿Por qué esta tabla tiene esa clave primaria?
- ¿Qué relación existe entre estas tablas?
- ¿Qué sucede cuando eliminás este registro?
- ¿Dónde se está realizando realmente la persistencia?

HTTP/API

- ¿Qué diferencia hay entre GET y POST?
- ¿Por qué utilizaron este endpoint?
- ¿Qué significa un HTTP 404?
- ¿Qué información viaja en el request?
- ¿Qué devuelve el servidor?
- ¿Qué formato de archivo usamos y por qué no otro?

Diseño

- ¿Qué patrón están utilizando?
- ¿Por qué eligieron ese patrón?
- ¿Qué problema solucionaría?
- ¿Podrían haberlo hecho de otra manera?
- ¿Qué ventaja aporta esta decisión?